Relatório do Projeto:
TP1 AL2 - Busca por Entregadores em BH

Autor: Tomás Moreira - 2024066113

1. Introdução

O presente trabalho prático detalha a concepção e implementação de um sistema de despacho e roteamento georreferenciado voltado para a distribuição de entregadores na região de Belo Horizonte. O objetivo principal da aplicação é localizar os profissionais de entrega mais próximos a restaurantes e bares específicos, otimizando o tempo de resposta logístico. Ao contrário de soluções tradicionais que dependem de APIs externas de mapeamento, o sistema foi arquitetado para operar localmente. Para isso, a aplicação conta com um backend robusto em Python responsável pelo processamento dos dados geográficos e uma interface de usuário interativa construída com HTML, CSS e JavaScript. A renderização espacial da malha viária vetorial ocorre integralmente no lado do cliente (via Canvas), garantindo alta performance e independência de Redes de Distribuição de Conteúdo (CDNs) ou serviços web de terceiros.

2. Implementação e Processamento de Dados

O fluxo de execução do programa inicia-se com a ingestão e tratamento de bases de dados externas, sendo dividido em duas frentes principais: os dados de atividades econômicas e a malha viária.

2.1. Parsing e Filtragem dos Estabelecimentos

A base de dados de entrada é composta por um arquivo CSV contendo os registros de atividades econômicas da cidade. O módulo de leitura realiza a extração (parsing) desses registros, aplicando filtros rigorosos para manter no sistema apenas os locais cujas atividades correspondam ao escopo do projeto (restaurantes e bares). Devido à natureza dos dados brutos, as coordenadas geográficas originais encontram-se projetadas no sistema UTM (EPSG:31983). Para padronizar a indexação espacial, o sistema emprega a biblioteca pyproj para realizar a conversão computacional dessas coordenadas para o formato padrão de latitude e longitude (EPSG:4326). Durante esta etapa, o algoritmo verifica as fronteiras geográficas de interesse e descarta sumariamente os estabelecimentos que se encontram fora da bounding box predefinida, otimizando o consumo de memória.

2.2. Construção do Grafo Viário e Alocação da Frota

Para a representação do ambiente urbano, o sistema processa um arquivo no formato GraphML fornecido pelo OpenStreetMap. O algoritmo percorre a estrutura de dados topológica filtrando as arestas do grafo. Vias destinadas exclusivamente a pedestres, ciclovias, escadarias e áreas de acesso restrito são removidas da estrutura em memória. Como resultado, preserva-se um subgrafo direcionado contendo unicamente as vias trafegáveis por veículos motorizados.

A geração da frota sintética ocorre de forma estocástica. Para cada entregador instanciado, a posição inicial é definida por meio de um sorteio que segue uma distribuição normal, tendo como média as coordenadas dos estabelecimentos validados. Para garantir a coerência física da simulação, a coordenada bruta sorteada é submetida a uma função de ajuste que projeta o entregador no nó (vértice) válido mais próximo pertencente ao grafo viário.

3. Estruturas de Dados e Indexação Espacial

O núcleo de eficiência do roteamento se da na implementação de uma estrutura de indexação espacial própria descartando varreduras lineares custosas.

3.1. Codificação Geohash

Para mapear as coordenadas bidimensionais em um espaço de busca unidimensional, o sistema converte a latitude e a longitude de cada membro da frota em um código alfanumérico designado como Geohash. A resolução estipulada é de até oito caracteres. A função de codificação processa as coordenadas baseando em verificações lógicas de ponto médio e mapeia o resultado em uma string formato Base32. Essa característica assegura que entregadores muito próximos na vida real compartilhem um longo prefixo comum em suas respectivas codificações.

3.2. Fórmulas Matemáticas e Limites de Busca

Para evitar buscas exaustivas, o sistema calcula as fronteiras de uma caixa envolvente imaginária ao redor da origem escolhida pelo usuário. A aproximação geométrica de latitude e longitude no raio exigido é calculada pelas seguintes equações de proporção esférica:

- Variação de Latitude: $\Delta lat = \frac{R}{110.574}$
- Variação de Longitude: $\Delta lon = \frac{R}{111.320 \cdot \cos(lat_{origem})}$

Após isolar os prefixos contidos nessa região, o algoritmo calcula a métrica de Haversine para atestar com precisão a distância real de todos os candidatos aprovados no índice. A equação trigonométrica utilizada é expressa por:

$$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

Onde r representa o raio métrico terrestre médio, ϕ representa as latitudes radiais avaliadas e λ denota as variáveis de longitude envolvidas no cálculo individual.

3.3. Arquitetura da Trie Radix

O agrupamento das chaves é processado estruturalmente por uma Radix Trie implementada internamente. Diferente das árvores convencionais, a estrutura Trie particiona os prefixos em suas arestas. O vértice raiz atua como um ponto vazio e as ramificações descendentes empregam dicionários para mapear os caracteres até os próximos nós limitadores. A implementação condensa múltiplos trechos únicos de caracteres dentro de uma mesma tupla para poupar recursos computacionais. Cada vértice terminal abriga as referências da classe do entregador mapeado na coordenada.

3.4. Análise de Complexidade Assintótica

A performance da estrutura de indexação respeita a métrica de complexidade de tempo baseando no tamanho máximo da chave estabelecida L, no prefixo de busca P e na quantia de elementos correspondentes extraídos K.

- **Inserção na Trie:** $O(L)$. O tamanho absoluto de 8 caracteres assegura um percurso iterativo constante resultando em um tempo delimitado estritamente por $O(1)$.
- **Busca por Prefixo:** $O(P + K)$. O caminho percorrido nas tuplas consome um tempo equivalente a P seguido pela coleta recursiva atrelada linearmente ao montante K de folhas conectadas.
- **Consulta Completa com Distância:** $O(C \cdot (P + K) + K \log K)$. A avaliação dos limitadores envolventes C possui tempo controlável. O filtro Trie captura os K candidatos. A triagem com a função trigonométrica Haversine e a respectiva ordenação final constituem o principal fator de peso representado pelo termo $K \log K$.

4. Interface e Integração Backend

A camada de interação com o usuário foi arquitetada sob a premissa de máxima independência de frameworks de terceiros. A solução garante que o processamento topológico pesado ocorra de forma otimizada no lado do servidor enquanto a exibição gráfica fluida seja delegada estritamente ao navegador web do cliente.

4.1. Servidor Nativo e Concorrência

O backend dispensa ferramentas externas e opera exclusivamente sobre as bibliotecas padrão da linguagem Python. O sistema instancia um servidor local empregando a classe

ThreadingHTTPServer em conjunto com tratadores customizados estendidos a partir da fundação BaseHTTPRequestHandler. Essa abordagem viabiliza o recebimento e o processamento de múltiplas requisições simultâneas através de rotas de API de forma concorrente. O módulo de serviços utiliza travas de exclusão mútua (RLock) para proteger o acesso à memória compartilhada durante o recálculo estocástico da frota e as inserções na árvore Radix.

4.2. Renderização Espacial no Cliente

A interface visual foi construída empregando unicamente HTML5, CSS3 e JavaScript puro. A renderização integral do ambiente urbano e dos marcadores ocorre no navegador através da manipulação direta da API do elemento Canvas. Para projetar a malha viária vetorial na tela, o frontend executa rotinas matemáticas próprias de projeção geométrica. O script processa os vértices originais em latitude e longitude e os converte para coordenadas cartesianas baseando em equações de translação e escala proporcionais à área visível da tela.

Para assegurar alto desempenho e contornar gargalos clássicos de desenho, a arquitetura introduz uma técnica de desenho em plano de fundo. A infraestrutura estática de ruas é desenhada uma única vez em um buffer de memória oculto. Durante interações contínuas de arrasto lateral ou mudança de magnificação executadas pelo usuário, a aplicação não redesenha as arestas individuais. O sistema apenas recopia este buffer preexistente com as transformações matriciais adequadas e plota os marcadores dinâmicos da frota sobre a imagem estática.

5. Testes Automatizados e Validação

A confiabilidade do código e a correção dos algoritmos foram asseguradas por meio de uma suíte abrangente de testes de unidade implementada com a biblioteca padrão unittest do Python. O conjunto de testes foi desenvolvido visando cobrir os módulos críticos do sistema.

5.1. Cobertura de Componentes

- **Tratamento de Dados:** Os testes do módulo de importação validam o carregamento de arquivos CSV, atestando a filtragem correta dos estabelecimentos de acordo com seus códigos CNAE e a conversão das coordenadas geográficas UTM para graus geográficos originais, além de verificar o comportamento da leitura do banco de dados previamente preparado.

- **Índice Espacial (Geohash e Trie):** O algoritmo da codificação Geohash foi validado contra casos predefinidos de codificação e decodificação, garantindo que as áreas de abrangência delimitadoras (bounds) correspondam corretamente ao valor projetado. A estrutura Trie teve sua segurança atestada por meio de asserções que validam as divisões parciais das arestas e o tratamento de colisões e chaves sobrepostas.
- **Lógica de Proximidade:** O cálculo matemático de Haversine foi submetido a cenários ideais testando casos limite de distâncias. Adicionalmente, o módulo de consulta simulado do índice avalia a exclusão sistemática de entregadores alocados fora do raio definido.
- **Módulo de Simulação Viária:** A filtragem da rede GraphML viária offline possui asserções específicas para assegurar que calçadas, escadarias e vias privadas sejam estritamente descartadas do grafo resultante. A função geradora da frota foi submetida a sementes controladas assegurando que os novos nós caiam perfeitamente sobre as ruas aprovadas.
- **Verificação de Serviços da Interface:** Os endpoints nativos do roteador web possuem casos de testes injetados com chamadas simuladas de clientes, confirmando que buscas inválidas, limitações de parâmetros e ausências de estado retornem os códigos HTTP com a estrutura correta de objeto JSON encapsulada na carga útil de resposta.

6. Conclusão

A construção deste trabalho prático apresentou desafios técnicos significativos que enriqueceram todo o processo construtivo de aprendizado da disciplina. A primeira grande dificuldade encontrada residiu na mineração de registros brutos. O processo de descobrir uma base governamental alternativa e criar um longo duto seguro de conversão topológica para extrair arquivos do BBBike até obter o grafo GraphML limpo exigiu extensas horas de pesquisa. O segundo obstáculo expressivo concentrou esforços na implementação puramente nativa da estrutura Radix Trie. Codificar essa árvore do zero exigiu pleno entendimento sobre a divisão de índices em ponteiros de memória visando comprimir eficientemente os fragmentos Geohash. A terceira e última dificuldade notória surgiu durante a transição da interface gráfica de consulta. Nas primeiras iterações do código o mapa era inteiramente gerado em janelas nativas escritas em Python. Porém, a visualização local sofria com lentidão severa e o aspecto estético comprometia o uso. A necessidade forçou o aprendizado emergencial de ferramentas voltadas para o desenvolvimento web. Manipular as coordenadas e aprender as regras de redimensionamento do Canvas em conjunto com as marcações HTML

revelou ser o gargalo mais demorado do trajeto. Contudo, todos os entraves foram suplantados, originando um painel veloz e um roteador que superou plenamente os prazos teóricos requisitados pelo escopo do projeto.

7. Referências Bibliográficas

[1] CORMEN, Thomas H. et al. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier.
[2] VIMIEIRO, Renato. Material didático e slides de estudos da disciplina de Alg2.
[3] BBBIKE. Extract service for OpenStreetMap. Disponível para extração local de mapas vetoriais brutos.
[4] OPENSTREETMAP. Base global de dados cartográficos livres e malhas topológicas mundiais.
[5] NETWORKX. Software for Complex Networks. Referência oficial e documentação arquitetural da biblioteca estrutural de grafos.